

Executive PG Diploma — Machine Learning

PREDICTED END-SEMESTER SAMPLE EXAM

(Practice paper modelled on 29-Nov-2025 actual paper)

Duration: 3 hours Max Marks: 100

Instructions: Handwritten and printed slides/notes permitted. Show all working.

Topic coverage:

*True/False (concepts) • Robust statistics • Imbalanced metrics • Outlier sensitivity
Perceptron • Hard-margin SVM • Gradient descent • Mini-batch GD
Logistic regression • GDA • Naive Bayes • Bagging vs Boosting
Decision trees • K-Means • Ridge/Lasso • PCA pipeline*

Sanjay | EPGD AI & DS, IIT Bombay — 2026

Marks Distribution

Q#	Topic	Marks
1	5 True/False with justification (1×5)	5
2	Robust regression: mean vs median	4
3	Cost-sensitive metric for imbalanced data	4
4	Outlier sensitivity ranking	4
5	Perceptron + Hard-margin SVM (3 parts)	15
6	Gradient descent on a quadratic	10
7	Batch vs Mini-batch GD	6
8	Logistic regression: prediction + cost	7
9	GDA: likelihoods + posterior	8
10	Naive Bayes on tennis dataset	8
11	Bagging vs Boosting	8
12	PCA full pipeline (10 sub-parts)	20
	TOTAL	100

Q1. Indicate whether each statement is TRUE or FALSE. Justify your answer in one or two lines.
[1×5 = 5 marks]

(a) In logistic regression, the decision boundary is always non-linear in the original features.

Solution:

✓ **ANSWER:** FALSE. The decision boundary is $\theta^T x = 0$ — a LINEAR hyperplane in the input features. Non-linearity only arises if we add polynomial / kernel features.

(b) Ridge regression can drive coefficients to exactly zero, performing feature selection.

Solution:

✓ **ANSWER:** FALSE. Ridge (L2) shrinks all coefficients smoothly toward zero but rarely hits exactly zero. Only Lasso (L1) produces sparse solutions.

(c) Increasing K in K-Nearest-Neighbours generally reduces variance and increases bias.

Solution:

✓ **ANSWER:** TRUE. Larger K averages over more neighbours → predictions become smoother (lower variance) but blur class boundaries (higher bias).

(d) In AdaBoost, a weak classifier with error $\epsilon = 0.5$ receives a weight $\alpha = 0$ and is effectively ignored.

Solution:

✓ **ANSWER:** TRUE. $\alpha_t = \frac{1}{2} \ln((1-\epsilon)/\epsilon)$. At $\epsilon = 0.5$: $\alpha = \frac{1}{2} \ln(1) = 0$, so the classifier contributes nothing to the final vote.

(e) K-Means clustering will always converge to the GLOBAL minimum of the within-cluster sum of squares.

Solution:

✓ **ANSWER:** FALSE. K-Means converges in finite steps, but only to a LOCAL minimum. The result depends on initialisation. Use K-Means++ or multiple restarts (n_{init}).

Q2. Given a dataset of M scalar values, you wish to find a single representative number (i.e. fit $y = c$ independent of any feature). Suggest an appropriate estimator for each case and justify. [4 marks]

(a) Data without outliers. [2]

Solution:

Step 1: Set up the regression problem.

Minimise $J(c) = \sum_i (y_i - c)^2$ (mean squared error).

Step 2: Set derivative to zero.

$$dJ/dc = -2 \sum_i (y_i - c) = 0 \Rightarrow c = (1/M) \sum y_i.$$

✓ **ANSWER:** Use the MEAN: $c^* = (1/M) \sum y_i$. Optimal under squared loss when data is clean.

(b) Data with outliers. [2]

Solution:

Step 1: Mean is sensitive to outliers — they pull it disproportionately.

Step 2: Use absolute-error loss $J(c) = \sum |y_i - c|$.

Step 3: Minimiser of L1 loss is the MEDIAN of the data.

✓ **ANSWER:** Use the MEDIAN — robust to outliers because each point contributes ± 1 to the gradient regardless of magnitude.

? **KEY IDEA:** Mean minimises L2 error (sum of squares). Median minimises L1 error (sum of absolute deviations). That's why median is robust.

Q3. A medical diagnostic test is built on 10,000 samples, of which only 80 are positive (rare disease). Propose a weighted classification error metric for this imbalanced problem and justify each choice. *[4 marks]*

Solution:

Step 1: Identify the imbalance.

Positives = 80 (1%); Negatives = 9920 (99%). Naïve accuracy is misleading — a 'predict all negative' model gets 99.2% accuracy but misses every sick patient.

Step 2: Propose weighted error.

$$E = (1/N) \cdot \sum_i w_i \cdot 1\{y_i \neq h(x_i)\}$$

Where w_i depends on the true class:

- $w = 1$ for false POSITIVES (predict disease when healthy)
- $w = 124$ for false NEGATIVES (miss a sick patient)

Step 3: Choose the weight ratio.

Choose $w_{FN} / w_{FP} = N_{neg} / N_{pos} \approx 9920/80 \approx 124$. This balances the contribution of the two classes in the loss.

Step 4: Justify clinically.

Missing a sick patient is far costlier than a false alarm; the weight reflects this asymmetric cost.

✓ **ANSWER:** Weighted error with $w_{FN} = 124$, $w_{FP} = 1$. Sample positives 124× more heavily so the model has a strong incentive to detect the rare positive class.

? **KEY IDEA:** Equivalent approaches: oversampling minority class, undersampling majority, SMOTE, or `class_weight='balanced'` in sklearn.

Q4. A high-leverage outlier (far from other x 's AND far from the original regression line) is added to a dataset. Four regression methods are re-fitted: (i) Logistic Regression, (ii) Ridge Regression, (iii) Decision Tree Regression, (iv) Random Forest Regression. Rank them in DECREASING influence of the outlier and justify. **[4 marks]**

Solution:

Step 1: Analyse each method.

(i) Logistic Regression (here assumed as least-squares fit on logits):

Quadratic loss $\rightarrow$ the squared residual at the outlier is HUGE $\rightarrow$ drags the entire line. Rank 1 (most affected).

(ii) Ridge Regression:

Still uses squared loss, but the L2 penalty on β resists large coefficient changes. Less affected than plain OLS / linear, but still moves. Rank 2.

(iii) Decision Tree Regression:

Only the LEAF containing the outlier is affected; predictions for other x -regions are unchanged. Rank 3.

(iv) Random Forest Regression:

Bootstrap aggregation: only $\sim 63\%$ of trees see the outlier, and averaging dilutes its effect further. Rank 4 (least affected).

Step 2: Final ranking.

✓ **ANSWER:** Decreasing influence: Logistic Reg. > Ridge > Decision Tree > Random Forest.

? **KEY IDEA:** Methods using squared loss are most outlier-sensitive. Tree-based methods are inherently robust because they partition space — outliers stay in their own leaf.

Q5. Consider the following 2-D dataset. [5+5+5 = 15 marks]

Feature	Pt1	Pt2	Pt3	Pt4	Pt5	Pt6
X_1	1	2	3	6	7	8
X_2	1	0	2	5	4	6
y	+1	+1	+1	-1	-1	-1

(a) Draw a 2-D plot of the data and obtain an equation of any hyperplane that correctly classifies these points. Show the weight vector. [5]

Solution:

Step 1: Plot the points mentally — class +1 is in the bottom-left, class -1 in the top-right.

Step 2: Try the simple line $x_1 + x_2 = 5$ (midway between the two groups).

Step 3: Check each point.

Pt1: $1+1 = 2 \leq 5 \Rightarrow$ side A (label +1) ✓

Pt2: $2+0 = 2 \leq 5 \Rightarrow$ side A ✓

Pt3: $3+2 = 5$ — on the line (push to 4.5 to be safe).

Pt4: $6+5 = 11 > 5 \Rightarrow$ side B (label -1) ✓

Pt5: $7+4 = 11 > 5 \Rightarrow$ side B ✓

Pt6: $8+6 = 14 > 5 \Rightarrow$ side B ✓

Step 4: Write in standard form $w^T x + b = 0$.

Define $w = (-1, -1)$, $b = 5$ (so for +1 class $w^T x + b > 0$).

Check signs: for Pt1, $(-1)(1) + (-1)(1) + 5 = 3 > 0 \rightarrow$ predict +1 ✓.

✓ **ANSWER:** Hyperplane: $-x_1 - x_2 + 5 = 0$, equivalently $x_1 + x_2 = 5$. Weight vector $w = (-1, -1)$, $b = 5$.

🔗 **KEY IDEA:** Any scalar multiple of w (and matching b) is a valid solution. We chose $(-1, -1)$ for simplicity.

(b) Given $w_1 = 0.7$, $w_2 = 0.5$, $w_0 = -1$, perform one perceptron update on point Pt4 = (6, 5), $y = -1$. [5]

Solution:

Step 1: Compute current prediction.

$$\text{score} = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 = -1 + 0.7 \cdot 6 + 0.5 \cdot 5 = -1 + 4.2 + 2.5 = 5.7$$

$$\hat{y} = \text{sign}(5.7) = +1.$$

Step 2: Check error.

True $y = -1$, predicted $+1 \rightarrow$ MISCLASSIFIED. Apply update.

Step 3: Perceptron update rule (with $\eta = 1$): $w \leftarrow w + \eta \cdot y \cdot x$.

$$w_0 \leftarrow -1 + 1 \cdot (-1) \cdot 1 = -2$$

$$w_1 \leftarrow 0.7 + 1 \cdot (-1) \cdot 6 = -5.3$$

$$w_2 \leftarrow 0.5 + 1 \cdot (-1) \cdot 5 = -4.5$$

Step 4: Verify Pt4 is now correctly classified.

$$\text{new score} = -2 + (-5.3) \cdot 6 + (-4.5) \cdot 5 = -2 - 31.8 - 22.5 = -56.3 \rightarrow \hat{y} = \text{sign}(-56.3) = -1 \quad \checkmark$$

✓ **ANSWER:** $w_t = (w_0, w_1, w_2) = (-1, 0.7, 0.5) \rightarrow w_{t+1} = (-2, -5.3, -4.5)$.

⚠ **WATCH OUT:** Many students forget to include the bias term in the update. Use the augmented $x = (1, x_1, x_2)$ so $x_0 = 1$.

(c) Write the hard-margin SVM primal optimisation for this data and state how many support vectors exist. Justify briefly. [5]

Solution:

Step 1: Write the primal.

$$\min \frac{1}{2}(w_1^2 + w_2^2) \quad \text{s.t.} \quad y_i(w_1 \cdot x_{1i} + w_2 \cdot x_{2i} + b) \geq 1 \quad \text{for } i = 1, \dots, 6$$

Step 2: List the 6 constraints explicitly.

$$(+1)(w_1 \cdot 1 + w_2 \cdot 1 + b) \geq 1 \quad [\text{Pt1}]$$

$$(+1)(w_1 \cdot 2 + w_2 \cdot 0 + b) \geq 1 \quad [\text{Pt2}]$$

$$(+1)(w_1 \cdot 3 + w_2 \cdot 2 + b) \geq 1 \quad [\text{Pt3}]$$

$$(-1)(w_1 \cdot 6 + w_2 \cdot 5 + b) \geq 1 \quad [\text{Pt4}]$$

$$(-1)(w_1 \cdot 7 + w_2 \cdot 4 + b) \geq 1 \quad [\text{Pt5}]$$

$$(-1)(w_1 \cdot 8 + w_2 \cdot 6 + b) \geq 1 \quad [\text{Pt6}]$$

Step 3: Identify support vectors geometrically.

The closest $+1$ point to the optimal boundary is Pt3 = (3, 2).

The closest -1 point is Pt4 = (6, 5).

These two points lie EXACTLY on the margin $\rightarrow$ they are the support vectors.

✓ **ANSWER:** 2 support vectors: Pt3 = (3, 2) and Pt4 = (6, 5).

🔑 **KEY IDEA:** In hard-margin SVM, only the 'closest' points from each class touch the margin. For 2-D linearly separable data, typically you'll have 2-3 SVs.

Q6. Consider $f(x) = (x - 6)^2$. Starting at $x_0 = 10$ with learning rate $\eta = 0.2$, do gradient descent. [4 + 6 = 10 marks]

(a) Compute the gradient at $x_0 = 10$. [4]

Solution:

Step 1: Differentiate f .

$$f(x) = (x - 6)^2$$

$$f'(x) = 2(x - 6)$$

Step 2: Evaluate at $x = 10$.

$$f'(10) = 2 \cdot (10 - 6) = 2 \cdot 4 = 8.$$

✓ **ANSWER:** Gradient at $x_0 = 10$ is 8.

(b) Compute the updated x after one and two iterations. [6]

Solution:

Step 1: Gradient descent rule.

$$x_{k+1} = x_k - \eta \cdot f'(x_k)$$

Step 2: Iteration 1.

$$x_1 = 10 - 0.2 \cdot 8 = 10 - 1.6 = 8.4$$

Step 3: Iteration 2.

$$f'(8.4) = 2 \cdot (8.4 - 6) = 4.8$$

$$x_2 = 8.4 - 0.2 \cdot 4.8 = 8.4 - 0.96 = 7.44$$

✓ **ANSWER:** After 1 step: $x_1 = 8.4$. After 2 steps: $x_2 = 7.44$.

🔗 **KEY IDEA:** True minimum is $x^* = 6$. After each step we're getting closer. The 'gradient descent shrinks the residual by factor $(1 - 2\eta)$ ' is a useful intuition (for η small).

Q7. Suppose you have a dataset with 5000 samples. *[2+2+2 = 6 marks]*

(a) In batch gradient descent, how many samples are used per iteration? [2]

Solution:

✓ **ANSWER:** Batch GD uses the ENTIRE dataset per iteration → 5000 samples per step.

(b) In mini-batch GD with batch size 100, how many samples per iteration? [2]

Solution:

✓ **ANSWER:** 100 samples per iteration.

(c) After 50 iterations of mini-batch GD with batch size 100 (with replacement), how many total sample usages? [2]

Solution:

Step 1: Each iteration uses 100 samples; total iterations = 50.

Step 2: Total = $50 \times 100 = 5000$.

✓ **ANSWER:** 5000 total sample usages.

? **KEY IDEA:** This equals one full pass (1 epoch) over a 5000-sample dataset.

Q8. Consider logistic regression with $h_{\theta}(x) = 1/(1 + e^{(-\theta^T x)})$, parameter vector $\theta = (-1, 0.5, 2)$, and input $x = (1, 2, 1)$ (bias term included). True label $y = 0$. [3 + 4 = 7 marks]

(a) Compute $h_{\theta}(x)$. [3]

Solution:

Step 1: Compute the linear score.

$$\theta^T x = (-1) \cdot 1 + 0.5 \cdot 2 + 2 \cdot 1 = -1 + 1 + 2 = 2.$$

Step 2: Apply sigmoid.

$$h_{\theta}(x) = 1/(1 + e^{(-2)}) = 1/(1 + 0.1353) = 1/1.1353 \approx 0.881.$$

✓ **ANSWER:** $h_{\theta}(x) \approx 0.881$.

(b) Compute the cross-entropy cost for this single example. [4]

Solution:

Step 1: Write the per-sample cost.

$$J = -[y \cdot \log(h) + (1 - y) \cdot \log(1 - h)]$$

Step 2: Substitute $y = 0$.

$$\begin{aligned} J &= -[0 \cdot \log(0.881) + 1 \cdot \log(1 - 0.881)] \\ &= -\log(0.119) \end{aligned}$$

Step 3: Compute log.

$$\log(0.119) \approx -2.129 \text{ (natural log)}$$

$$J \approx +2.129.$$

✓ **ANSWER:** Cost $J \approx 2.13$ (high, because the model predicted 0.881 when true label is 0).

🔗 **KEY IDEA:** Cross-entropy heavily penalises confident wrong predictions. $h = 0.881$ with $y = 0$ means 'the model said 88% chance of 1 but it's actually 0' — huge penalty.

Q9. Binary classification with one continuous feature $x \in \mathbb{R}$ modelled by GDA. Class-conditional densities are $p(x | y = 0) = N(\mu=2, \sigma^2=1)$ and $p(x | y = 1) = N(\mu=5, \sigma^2=1)$. Priors: $P(y=1) = 0.3$, $P(y=0) = 0.7$. Observed $x = 4$. [4 + 4 = 8 marks]

(a) Compute the likelihoods $p(x = 4 | y = 0)$ and $p(x = 4 | y = 1)$. [4]

Solution:

Step 1: Gaussian density.

$$p(x | \mu, \sigma^2) = (1/\sqrt{2\pi\sigma^2}) \cdot \exp(-(x-\mu)^2/(2\sigma^2))$$

Step 2: Compute $p(x=4 | y=0)$: $\mu=2, \sigma^2=1$.

$$(x-\mu)^2 = (4-2)^2 = 4$$

$$\text{Exponent} = -4/2 = -2$$

$$\text{Normaliser} = 1/\sqrt{2\pi} \approx 0.3989$$

$$p(x=4 | y=0) = 0.3989 \cdot e^{-2} = 0.3989 \cdot 0.1353 \approx 0.0540.$$

Step 3: Compute $p(x=4 | y=1)$: $\mu=5, \sigma^2=1$.

$$(x-\mu)^2 = (4-5)^2 = 1$$

$$\text{Exponent} = -1/2 = -0.5$$

$$p(x=4 | y=1) = 0.3989 \cdot e^{-0.5} = 0.3989 \cdot 0.6065 \approx 0.2420.$$

✓ **ANSWER:** $p(x=4 | y=0) \approx 0.0540$ and $p(x=4 | y=1) \approx 0.2420$.

(b) Compute the posterior $P(y = 1 | x = 4)$. [4]

Solution:

Step 1: Apply Bayes' rule.

$$P(y=1|x) = p(x|y=1) \cdot P(y=1) / [p(x|y=0) \cdot P(y=0) + p(x|y=1) \cdot P(y=1)]$$

Step 2: Numerator = $0.2420 \cdot 0.3 = 0.0726$.

Step 3: Denominator = $0.0540 \cdot 0.7 + 0.2420 \cdot 0.3 = 0.0378 + 0.0726 = 0.1104$.

Step 4: $P(y=1 | x=4) = 0.0726 / 0.1104 \approx 0.657$.

✓ **ANSWER:** $P(y=1 | x=4) \approx 0.657 \rightarrow$ predict class 1.

🔑 **KEY IDEA:** Even though the prior favours class 0 (0.7 vs 0.3), the likelihood ratio is so strong that the posterior flips to class 1. Likelihood $\times$ prior matters, not prior alone.

Q10. Dataset of 6 emails. Features: 'free' (Yes/No), 'urgent' (Yes/No). Label: Spam (Y/N). [3 + 5 = 8 marks]

#	free	urgent	Spam
1	Yes	Yes	Y
2	Yes	No	Y
3	No	Yes	Y
4	No	No	N
5	Yes	No	N
6	No	No	N

(a) Explain the naive independence assumption and state one limitation. [3]

Solution:

The assumption: given the class label, the features are conditionally independent:

$$P(\text{free, urgent} \mid \text{Spam}) = P(\text{free} \mid \text{Spam}) \cdot P(\text{urgent} \mid \text{Spam})$$

This means once we know an email is spam, the presence of 'free' tells us nothing extra about whether 'urgent' also appears.

Limitation:

Real features are usually correlated (spam emails often use BOTH 'free' AND 'urgent' together). The assumption is rarely literally true, so probability estimates can be biased — but classification rankings often still come out right.

✓ **ANSWER:** Conditional independence simplifies the joint likelihood into a product. Limitation: it's usually false, so probability estimates can be poorly calibrated.

(b) Compute $P(\text{Spam} = Y \mid \text{free} = \text{Yes}, \text{urgent} = \text{No})$ and predict the class. [5]

Solution:

Step 1: Class priors.

$$P(Y) = 3/6 = 0.5, \quad P(N) = 3/6 = 0.5.$$

Step 2: Likelihoods given class Y.

Spam=Y has emails {1, 2, 3}. 'free=Yes' in emails 1, 2 $\rightarrow 2/3 \approx 0.667$. 'urgent=No' in email 2 $\rightarrow 1/3 \approx 0.333$.

Step 3: Likelihoods given class N.

Spam=N has emails {4, 5, 6}. 'free=Yes' in email 5 $\rightarrow 1/3 \approx 0.333$. 'urgent=No' in emails 4, 5, 6 $\rightarrow 3/3 = 1.0$.

Step 4: Compute unnormalised posteriors.

$$P(Y|x) \propto P(Y) \cdot P(\text{free}=\text{Yes}|Y) \cdot P(\text{urgent}=\text{No}|Y) = 0.5 \cdot 0.667 \cdot 0.333 \approx 0.111.$$

$$P(N|x) \propto P(N) \cdot P(\text{free}=\text{Yes}|N) \cdot P(\text{urgent}=\text{No}|N) = 0.5 \cdot 0.333 \cdot 1.0 \approx 0.167.$$

Step 5: Normalise.

$$\text{Sum} = 0.111 + 0.167 = 0.278.$$

$$P(Y|x) = 0.111 / 0.278 \approx 0.400.$$

$$P(N|x) = 0.167 / 0.278 \approx 0.600.$$

✓ **ANSWER:** $P(\text{Spam} = Y | \text{free}=\text{Yes}, \text{urgent}=\text{No}) \approx 0.40 \rightarrow$ predict NOT SPAM (class N).

⚠ **WATCH OUT:** Always multiply $P(\text{class})$ by ALL features — including the 'No' values. Don't ignore absent features.

Q11. Explain the differences between Bagging and Boosting. [4 + 4 = 8 marks]

(a) Main conceptual difference: how base models are trained and combined. [4]

Solution:

Bagging (Bootstrap Aggregation):

- Trains B base models INDEPENDENTLY in PARALLEL.
- Each model uses a different bootstrap sample (sampled with replacement) of the training data.
- Combines predictions by averaging (regression) or majority vote (classification).
- Primary effect: reduces VARIANCE of the final predictor.

Boosting (e.g., AdaBoost):

- Trains base models SEQUENTIALLY.
- Each new model focuses on the MISTAKES of the previous models — the misclassified samples receive higher weights.
- Final prediction is a WEIGHTED vote: $H(x) = \text{sign}(\sum \alpha_t \cdot h_t(x))$.
- Primary effect: reduces BIAS, builds a strong learner from many weak ones.

✓ **ANSWER:** Bagging = parallel independent base models, variance reduction. Boosting = sequential dependent base models, bias reduction.

(b) State one advantage and one limitation of each. [4]

Solution:

Method	Advantage	Limitation
Bagging	Reduces variance; very robust to overfitting; embarrassingly parallel.	Hardly reduces bias — if base learners are themselves biased, the ensemble is too.
Boosting	Can convert weak learners (just above chance) into a strong classifier; reduces bias.	Sequential, so harder to parallelise; sensitive to noisy data and outliers (they get reweighted ever higher).

✓ **ANSWER:** Bagging: low overfitting risk but limited bias reduction. Boosting: strong accuracy but sensitive to noise.

Q12. You are given the following 2-D dataset of 6 points. Carry out PCA, project onto 1 component, and explain. *[2×10 = 20 marks]*

Point	x_1	x_2
1	2.5	2.4
2	0.5	0.7
3	2.2	2.9
4	1.9	2.2
5	3.1	3.0
6	1.0	1.0

Carry out the PCA pipeline answering each part.

Part 1: Compute the mean of each feature. [2]

Solution:

Step 1: Add and divide by 6.

$$\bar{x}_1 = (2.5 + 0.5 + 2.2 + 1.9 + 3.1 + 1.0)/6 = 11.2/6 \approx 1.867$$

$$\bar{x}_2 = (2.4 + 0.7 + 2.9 + 2.2 + 3.0 + 1.0)/6 = 12.2/6 \approx 2.033$$

✓ **ANSWER:** Means: $\bar{x}_1 \approx 1.867$, $\bar{x}_2 \approx 2.033$.

Part 2: Compute the standard deviation of each feature (use 1/N). [2]

Solution:

Step 1: Deviations from mean for x_1 : 0.633, -1.367, 0.333, 0.033, 1.233, -0.867.

Squared: 0.401, 1.868, 0.111, 0.001, 1.521, 0.752 → sum = 4.654.

$$\text{Var}(x_1) = 4.654/6 \approx 0.776 \rightarrow \sigma_1 \approx 0.881.$$

Step 2: Deviations for x_2 : 0.367, -1.333, 0.867, 0.167, 0.967, -1.033.

Squared: 0.135, 1.778, 0.751, 0.028, 0.935, 1.067 → sum = 4.694.

$$\text{Var}(x_2) = 4.694/6 \approx 0.782 \rightarrow \sigma_2 \approx 0.884.$$

✓ **ANSWER:** $\sigma_1 \approx 0.881$, $\sigma_2 \approx 0.884$.

Part 3: Construct the centred matrix X_c (subtract the mean from each feature). [2]

Solution:

Pt	$x_1 - \bar{x}_1$	$x_2 - \bar{x}_2$
----	-------------------	-------------------

1	+0.633	+0.367
2	-1.367	-1.333
3	+0.333	+0.867
4	+0.033	+0.167
5	+1.233	+0.967
6	-0.867	-1.033

✓ **ANSWER:** Centred matrix obtained — each column now has mean ≈ 0 .

Part 4: Compute the 2x2 covariance matrix using $\Sigma = (1/N) X^{ct} X^c$. [2]

Solution:

Step 1: $\Sigma_{11} = \text{Var}(x_1) \approx 0.776$ (from Part 2).

Step 2: $\Sigma_{22} = \text{Var}(x_2) \approx 0.782$ (from Part 2).

Step 3: $\Sigma_{12} = (1/6) \sum (x_{1i} - \bar{x}_1)(x_{2i} - \bar{x}_2)$.

Products: $0.633 \cdot 0.367 + (-1.367)(-1.333) + 0.333 \cdot 0.867 + 0.033 \cdot 0.167 + 1.233 \cdot 0.967 + (-0.867)(-1.033)$

$$= 0.232 + 1.822 + 0.289 + 0.005 + 1.192 + 0.896 = 4.436$$

$$\Sigma_{12} = 4.436/6 \approx 0.739.$$

✓ **ANSWER:** $\Sigma = [[0.776, 0.739], [0.739, 0.782]]$.

Part 5: Compute the eigenvalues of Σ . [2]

Solution:

Step 1: Characteristic equation $\det(\Sigma - \lambda I) = 0$.

$$(0.776 - \lambda)(0.782 - \lambda) - 0.739^2 = 0$$

Step 2: Expand.

$$\lambda^2 - 1.558\lambda + 0.776 \cdot 0.782 - 0.546 = 0$$

$$\lambda^2 - 1.558\lambda + 0.607 - 0.546 = 0$$

$$\lambda^2 - 1.558\lambda + 0.061 = 0$$

Step 3: Apply quadratic formula.

$$\lambda = (1.558 \pm \sqrt{(1.558^2 - 4 \cdot 0.061)})/2 = (1.558 \pm \sqrt{(2.428 - 0.244)})/2 = (1.558 \pm \sqrt{2.184})/2$$

$$\sqrt{2.184} \approx 1.478.$$

$$\lambda = (1.558 \pm 1.478)/2 \rightarrow \lambda_1 \approx 1.518, \lambda_2 \approx 0.040.$$

✓ **ANSWER:** $\lambda_1 \approx 1.518$ (dominant), $\lambda_2 \approx 0.040$.

Part 6: Compute the corresponding eigenvectors. [2]

Solution:

Step 1: For $\lambda_1 = 1.518$: solve $(\Sigma - \lambda_1 I)v = 0$.

$$(0.776 - 1.518)v_1 + 0.739 \cdot v_2 = 0 \Rightarrow -0.742 v_1 + 0.739 v_2 = 0 \Rightarrow v_2 \approx v_1.$$

Eigenvector $v_1 \propto (1, 1)$. Normalise: $v_1 = (1/\sqrt{2}, 1/\sqrt{2}) \approx (0.707, 0.707)$.

Step 2: For $\lambda_2 = 0.040$: orthogonal to v_1 .

$$v_2 \approx (-1/\sqrt{2}, 1/\sqrt{2}) \approx (-0.707, 0.707).$$

✓ **ANSWER:** $v_1 \approx (0.707, 0.707)$ along $\lambda_1 = 1.518$; $v_2 \approx (-0.707, 0.707)$ along $\lambda_2 = 0.040$.

Part 7: How much variance does PC1 explain? Should we keep it alone? [2]

Solution:

Step 1: Variance ratio = $\lambda_1 / (\lambda_1 + \lambda_2)$.

$$= 1.518 / (1.518 + 0.040) = 1.518 / 1.558 \approx 0.974.$$

✓ **ANSWER:** PC1 captures ~97.4% of the total variance. We can confidently keep only PC1 and discard PC2.

Part 8: Form the projection matrix $W \in \mathbb{R}^{2 \times 1}$ for the top component. [2]

Solution:

✓ **ANSWER:** $W = [[0.707], [0.707]]$ — a 2×1 column vector containing v_1 .

Part 9: Project the centred data onto PC1 to get $z = X^c \cdot W \in \mathbb{R}^6$. [2]

Solution:

$$\text{Step 1: } z_i = 0.707 \cdot (x_{i1} - \bar{x}_1) + 0.707 \cdot (x_{i2} - \bar{x}_2).$$

$$\text{Step 2: Pt1: } 0.707 \cdot 0.633 + 0.707 \cdot 0.367 = 0.707 \cdot 1.000 = 0.707$$

$$\text{Pt2: } 0.707 \cdot (-1.367) + 0.707 \cdot (-1.333) = 0.707 \cdot (-2.700) = -1.909$$

$$\text{Pt3: } 0.707 \cdot 0.333 + 0.707 \cdot 0.867 = 0.707 \cdot 1.200 = 0.848$$

$$\text{Pt4: } 0.707 \cdot 0.033 + 0.707 \cdot 0.167 = 0.707 \cdot 0.200 = 0.141$$

$$\text{Pt5: } 0.707 \cdot 1.233 + 0.707 \cdot 0.967 = 0.707 \cdot 2.200 = 1.555$$

$$\text{Pt6: } 0.707 \cdot (-0.867) + 0.707 \cdot (-1.033) = 0.707 \cdot (-1.900) = -1.343$$

✓ **ANSWER:** $z = (0.707, -1.909, 0.848, 0.141, 1.555, -1.343)$.

Part 10: Plot z on a number line and comment on cluster structure. [2]

Solution:

Step 1: Sort z to see order: $-1.909, -1.343, 0.141, 0.707, 0.848, 1.555$.

Step 2: Visualisation: two natural groupings emerge — $\{-1.909, -1.343\}$ (Pts 2, 6) and $\{0.141, 0.707, 0.848, 1.555\}$ (Pts 1, 3, 4, 5).

✓ **ANSWER:** Projecting onto PC1 reveals 2 clusters: Pts 2,6 are 'small' (negative z) and Pts 1,3,4,5 are 'large' (positive z). PCA successfully captured the main spread direction.

🔑 **KEY IDEA:** PCA reduces dimensionality while preserving the directions of MAXIMUM VARIANCE. Use it as a preprocessing step before clustering, classification, or visualisation.

EXAM STRATEGY & LAST-MINUTE TIPS

Time Allocation Plan (3-hour exam)

Question	Time	Marks	Strategy
Q1 True/False	10 min	5	Quick wins — answer all confidently
Q2-Q4 Short concepts	20 min	12	1-2 lines + justification
Q5 Perceptron + SVM (3 parts)	30 min	15	Draw the plot first!
Q6-Q8 GD & Logistic	30 min	23	Compute step by step, show working
Q9 GDA	15 min	8	Use Gaussian formula + Bayes
Q10 Naive Bayes	20 min	8	Build prior + likelihood table first
Q11 Bagging vs Boosting	10 min	8	Definition + one example each
Q12 PCA (10 sub-parts)	40 min	20	Centre → Cov → Eig → Project
Review	15 min	—	Double-check arithmetic

Top 7 Mistakes to Avoid

1. Forgetting to include the bias term $x_0 = 1$ in $\theta^T x$ calculations.
2. Using wrong log base (exam uses $\log_2$ for entropy in bits; natural log for cross-entropy).
3. Not WEIGHTING child impurities by sample fractions in Information Gain.
4. Mixing up classification ($y \in \{0,1\}$) vs SVM ($y \in \{-1,+1\}$) labels in update rules.
5. Forgetting to STANDARDISE features for SVM, K-Means, Ridge, Lasso, PCA.
6. Confusing α regimes in SVM ($\alpha=0$ not-SV, $0<\alpha<C$ free-SV, $\alpha=C$ bound-SV).
7. Not normalising posterior probabilities (forgetting to divide by total).

Must-Recall Formulas

Topic	Formula
Sigmoid	$\sigma(z) = 1/(1+e^{-z})$
Cross-entropy	$J = -[y \log h + (1-y) \log(1-h)]$
Perceptron update	$w \leftarrow w + \eta \cdot y \cdot x$
SVM margin	$2/\ w\ $; $\min \frac{1}{2}\ w\ ^2$ s.t. $y_i(w^T x_i + b) \geq 1$
GD update	$x_{k+1} = x_k - \eta \cdot f'(x_k)$
GDA Gaussian PDF	$1/\sqrt{(2\pi\sigma^2)} \cdot \exp(-(x-\mu)^2/2\sigma^2)$
Bayes posterior	$P(y x) = P(x y)P(y)/\sum P(x y')P(y')$
NB MAP	$\hat{y} = \operatorname{argmax}_y P(y) \cdot \prod P(x_j y)$
AdaBoost α	$\alpha = \frac{1}{2} \ln((1-\epsilon)/\epsilon)$
Ridge β	$(X^T X + \lambda I)^{-1} X^T y$
Lasso soft-thresh	$\operatorname{sign}(\beta_{\text{OLS}}) \cdot \max(\beta_{\text{OLS}} - \lambda, 0)$
K-Means	$c(i) = \operatorname{argmin} \ x_i - \mu_k\ ^2$; $\mu_k = \operatorname{mean}(C_k)$
Information Gain	$IG = H(\text{parent}) - \sum w_v H(\text{child})$
Entropy / Gini	$H = -\sum p \log_2 p$; $G = 1 - \sum p^2$
PCA	Eigen-decompose Σ ; project on top eigenvectors

All the best, Sanjay! Go ace this exam!
